CS 432: Databases

FixIIT – Maintenance Portal

Assignment 2 - Module A

Submitted by: Team Codebase

Abhitej Singh Bhullar (23110009)
Shiv Patel (23110302)
Shivansh Shukla (23110303)
Soham Shrivastava (23110315)
Arin Mehta (23110038)

Github Link:
https://github.com/ArinMehta/FixIIT

Contents

1 Introduction **3**
1.1 Motivation: The B^+ Tree . . . 3
1.2 Project Scope . . . 3

2 Methodology **3**
2.1 B^+ Tree Node Design . . . 3
2.2 Insertion Methodology . . . 4
2.2.1 Split Strategy: Copy-up vs Push-up . . . 5
2.3 Deletion Methodology . . . 5
2.3.1 Rebalancing Decision Logic . . . 5
2.4 Range Query Methodology . . . 5
2.5 Table and Schema Validation . . . 6
2.6 Database Manager Architecture . . . 6

3 Functionality **7**
3.1 Core B^+ Tree Operations . . . 7
3.2 Table Layer . . . 7
3.3 DatabaseManager . . . 7
3.4 BruteForceDB (Baseline) . . . 7
3.5 Visualisation System . . . 7

4 Implementation Approach **8**
4.1 Design Decisions . . . 8
4.1.1 Proactive vs Reactive Splitting and Merging . . . 8
4.1.2 Upsert Semantics on Insert . . . 8
4.1.3 Separator Key Update on Delete . . . 8
4.1.4 Leaf Linked-List Maintenance . . . 8
4.2 File Structure . . . 8

5 Tree Visualisations **9**
5.1 Order-4 Tree (12 Keys) . . . 9
5.2 Order-6 Tree (30 Keys) . . . 9
5.3 Tree After Deletions . . . 10
5.4 fixiit_db Tickets Table Tree . . . 10

6 Database Manager Demo – fixiit_db **10**
6.1 Schema Overview . . . 10

7 Performance Analysis **11**
7.1 Experimental Setup . . . 11
7.2 Combined Performance Comparison . . . 11
7.3 Benchmark Results Table . . . 12
7.4 Speedup Analysis . . . 13
7.5 Memory Usage . . . 13
7.6 Log-Scale Growth Comparison . . . 14

7.7 Interpretation by Operation 14

8 Conclusion **15**
8.1 Summary of Findings 15
8.2 Key Findings 15
8.3 Challenges 15
8.4 Future Work 15

References **17**

1. Introduction

1.1. Motivation: The B$^+$ Tree

A **B$^+$ Tree** is a self-balancing, ordered tree data structure that solves this problem by guaranteeing **O(log n)** time for all point operations and **O(log n + k)** for range queries. It is the dominant indexing structure in relational databases (MySQL InnoDB, PostgreSQL, SQLite) and file systems (NTFS, ext4).

B$^+$ Tree – Formal Definition (Order m)

- Every internal node has at most m children ($m - 1$ keys).
- Every non-root node has at least $\lceil m/2 \rceil$ children.
- All actual data values are stored *only* in leaf nodes.
- Internal nodes store separator keys for routing only.
- Leaf nodes are linked in a singly-connected list for range queries.

1.2. Project Scope

Table 1: Project components and their responsibilities

Component	File	Responsibility
B$^+$ Tree	`bplustree.py`	Core index: insert, delete, search, range query
Table	`table.py`	Schema-validated record store over B$^+$ Tree
Database Manager	`db_manager.py`	Multi-database, multi-table manager
BruteForce DB	`bruteforce.py`	O(n) linear baseline for benchmarking
Performance	`performance.py`	Automated benchmarking and Matplotlib plots
Notebook	`report.ipynb`	Full demonstration and analysis

2. Methodology

2.1. B$^+$ Tree Node Design

Every node in the tree—whether internal or leaf—is represented by the `BPlusTreeNode` class. The class uses five attributes whose purpose depends on the node type.

```
class BPlusTreeNode:
    def __init__(self, order: int, is_leaf: bool = True):
```

```
        self.order    = order     # max children = order
        self.is_leaf  = is_leaf   # True -> leaf, False -> internal
        self.keys     = []        # sorted keys in this node
        self.values   = []        # (leaf only) data records
        self.children = []        # (internal only) child pointers
        self.next     = None      # (leaf only) next-leaf pointer
```

Listing 1: BPlusTreeNode class attributes

Table 2: Node attribute usage by node type

Attribute	Internal Node	Leaf Node	Purpose
`keys`	Yes	Yes	Sorted list of keys
`values`	–	Yes	Associated data records
`children`	Yes	–	Pointers to child nodes
`next`	–	Yes	Linked-list chain to next leaf
`is_leaf`	Yes	Yes	Node type flag

2.2. Insertion Methodology

Insertion uses a **proactive split** strategy: before descending into any child node, the algorithm checks if that child is already full ($|\text{keys}| = m-1$). If it is, the child is split *before* descent. This guarantees that when a leaf is reached, it always has space—eliminating the need for a second upward pass.

```
def _insert_non_full(self, node, key, value):
    if node.is_leaf:
        # Binary-search insertion; upsert if key exists
        i = 0
        while i < len(node.keys) and node.keys[i] < key:
            i += 1
        node.keys.insert(i, key)
        node.values.insert(i, value)
    else:
        i = 0
        while i < len(node.keys) and key >= node.keys[i]:
            i += 1
        if len(node.children[i].keys) == self.order - 1:
            self._split_child(node, i)    # split BEFORE descending
            if key >= node.keys[i]:
                i += 1
        self._insert_non_full(node.children[i], key, value)
```

Listing 2: Proactive split during insertion descent

2.2.1. Split Strategy: Copy-up vs Push-up

Table 3: Split behaviour for leaf vs internal nodes

Node Type	Mechanism	Key Fate
Leaf split	Cut at midpoint; right half becomes new leaf; update `next` pointers	**Copy-up**: median key appears in both parent and right leaf
Internal split	Cut at median; redistribute children	**Push-up**: median key moves to parent and is removed from both children

2.3. Deletion Methodology

Deletion uses a **proactive fill** strategy: before descending into any child, the algorithm ensures the child has *more than* `min_keys` keys. If the child is at minimum, `_fill_child` is called first.

```
def _fill_child(self, node, index):
    left  = node.children[index-1] if index > 0 else None
    right = node.children[index+1] if index < len(node.children)-1
        else None

    if left and len(left.keys) > self.min_keys:
        self._borrow_from_prev(node, index)   # rotate through parent
    elif right and len(right.keys) > self.min_keys:
        self._borrow_from_next(node, index)   # rotate through parent
    else:
        # Neither sibling can spare a key -> merge
        if right is not None:
            self._merge(node, index)
        else:
            self._merge(node, index - 1)
```

Listing 3: Proactive fill during deletion descent

2.3.1. Rebalancing Decision Logic

Table 4: Rebalancing strategy during deletion

Condition	Action	Complexity
Left sibling has a spare key	Borrow from left (rotate via parent)	O(1)
Right sibling has a spare key	Borrow from right (rotate via parent)	O(1)
Neither sibling has spare key	Merge siblings (pull separator down)	O(1)

2.4. Range Query Methodology

The leaf linked-list enables highly efficient range queries.

```python
def range_query(self, start_key, end_key) -> list:
    result = []
    leaf = self._find_leaf(self.root, start_key)  # O(log n)
    while leaf is not None:                        # walk forward
        for i, k in enumerate(leaf.keys):
            if k > end_key:
                return result
            if k >= start_key:
                result.append((k, leaf.values[i]))
        leaf = leaf.next                           # O(1) per step
    return result
    # Total complexity: O(log n + k) where k = result count
```

Listing 4: Range query using leaf linked-list

2.5. Table and Schema Validation

The `Table` class wraps a B^+ Tree with schema enforcement:

```python
def validate_record(self, record: dict) -> None:
    for col, type_str in self.schema.items():
        if col not in record:
            raise ValueError(f"Missing column '{col}'")
        expected_type = _TYPE_MAP.get(type_str, str)  # int/float/str/bool
        if not isinstance(record[col], expected_type):
            record[col] = expected_type(record[col])   # coerce or raise
```

Listing 5: Schema validation in Table class

2.6. Database Manager Architecture

The `DatabaseManager` maintains a two-level hierarchy: a dictionary of databases, each containing a dictionary of named `Table` objects. Each table is independently backed by its own B^+ Tree instance.

```
                    DatabaseManager
                          ↓
Database ‘‘fixiit_db’’   Database ‘‘other_db’’
                          ↓
     Table ‘‘tickets’’   Table ‘‘members’’
                ↓            ↓
   B+ Tree (order=8)   B+ Tree (order=8)
```

3. Functionality

3.1. Core B$^+$ Tree Operations

Table 5: B$^+$ Tree API – operations and time/space complexity

Method	Description	Time	Space
`insert(k, v)`	Insert key-value; upsert if exists	O(log n)	O(1)
`search(k)`	Return value for key; None if absent	O(log n)	O(1)
`delete(k)`	Remove key; rebalance tree	O(log n)	O(1)
`update(k, v)`	Overwrite value for existing key	O(log n)	O(1)
`range_query(a, b)`	All pairs where $a \leq k \leq b$	O(log n + k)	O(k)
`get_all()`	All pairs sorted by key	O(n)	O(n)
`visualize_tree()`	Graphviz Digraph of the tree	O(n)	O(n)

3.2. Table Layer

The `Table` class provides relational-style access on top of the B$^+$ Tree:

- **Schema enforcement:** column presence, type coercion, and extra-column warnings.
- **Search-key indexing:** any column can be designated as the B$^+$ Tree index key.
- **Full CRUD:** `insert`, `get`, `update`, `delete`, `range_query`, `get_all`.

3.3. DatabaseManager

- **Multi-database:** `create_database`, `delete_database`, `list_databases`.
- **Multi-table per database:** `create_table`, `delete_table`, `list_tables`, `get_table`.
- **Direct CRUD proxy:** `insert`, `get`, `update`, `delete`, `range_query`.
- **Describe:** pretty-prints database and table structure with row counts.

3.4. BruteForceDB (Baseline)

An unsorted Python list with a linear scan for every operation. All operations are O(n)—insert scans for duplicates, search scans all records, delete scans then shifts the array, range query scans then sorts. Used exclusively as a performance comparison baseline.

3.5. Visualisation System

The `visualize_tree()` method uses the Graphviz `Digraph` class with HTML-like table labels (not the `record` shape, which causes rendering failures on Windows) to produce the following output:

- **Internal nodes:** light-blue filled cells.
- **Leaf nodes:** light-green filled cells.
- **Parent-to-child edges:** solid black arrows.
- **Leaf next-pointer edges:** dashed dark-green arrows showing the linked list.

4. Implementation Approach

4.1. Design Decisions

4.1.1. Proactive vs Reactive Splitting and Merging

Table 6: Comparison of split/merge strategies

Strategy	Mechanism	Trade-off
Reactive	Split or merge on the way *back up* after descent	Requires a second upward traversal; simpler implementation
Proactive (chosen)	Split or merge on the way *down* before descent	No backtracking needed; may cause slightly more splits; chosen for clarity

4.1.2. Upsert Semantics on Insert

`insert(k, v)` checks for an existing key in the leaf before inserting. If found, the value is overwritten. This simplifies the update flow and matches real-world database `INSERT OR REPLACE` behaviour.

4.1.3. Separator Key Update on Delete

B^+ Trees store copies of leaf keys as separator keys in internal nodes. After a leaf deletion, the code scans the parent's key list and replaces any separator that equals the deleted key with the new minimum of the right subtree:

```
for j, k in enumerate(node.keys):
    if k == key:
        node.keys[j] = self._get_min_leaf_key(node.children[j + 1])
        break
```

Listing 6: Updating stale separator keys after deletion

4.1.4. Leaf Linked-List Maintenance

During every split and merge the `next` pointers are explicitly updated:

- **Leaf split:** new right node's `next` ← original `next`; original `next` ← new right node.
- **Leaf merge:** left `next` ← right `next` (the right node is discarded).

4.2. File Structure

```
db_management_system/
|-- database/
|   |-- __init__.py          # Package exports
|   |-- bplustree.py         # BPlusTreeNode + BPlusTree  (~300 lines)
|   |-- table.py             # Table with schema validation (~130
    lines)
```

```
|    |-- db_manager.py       # DatabaseManager              (~150 lines)
|    |-- bruteforce.py       # BruteForceDB baseline         (~80 lines)
|    `-- performance.py      # PerformanceAnalyzer           (~220 lines)
|-- report.ipynb             # Jupyter demonstration notebook
`-- requirements.txt         # graphviz, matplotlib, numpy
```

Listing 7: Project directory layout

5. Tree Visualisations

5.1. Order-4 Tree (12 Keys)

Figure 1 shows the tree after inserting 15 keys in random order. The blue internal node holds a separator key for routing. The green leaf nodes hold the actual data and are chained via dashed arrows representing the leaf linked list.

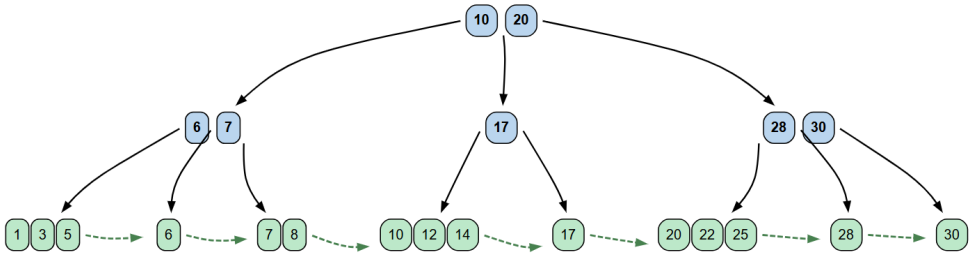

Figure 1: B^+ Tree (order=4, 12 keys) – blue = internal node, green = leaf node, dashed = next pointer

5.2. Order-6 Tree (30 Keys)

Figure 2 demonstrates the high branching factor of an order-6 tree. Despite 30 keys, the height is only 3 levels—each internal node routes up to 6 children simultaneously.

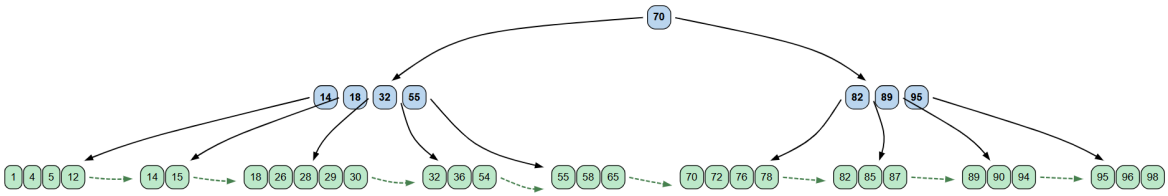

Figure 2: B^+ Tree (order=6, 30 keys) – height remains 3 levels due to high branching factor

5.3. Tree After Deletions

Figure 3 shows the order-4 tree after deleting keys 20, 5, and 17. The tree has rebalanced automatically through borrowing and merging. The leaf linked list remains unbroken and all B^+ Tree invariants are satisfied.

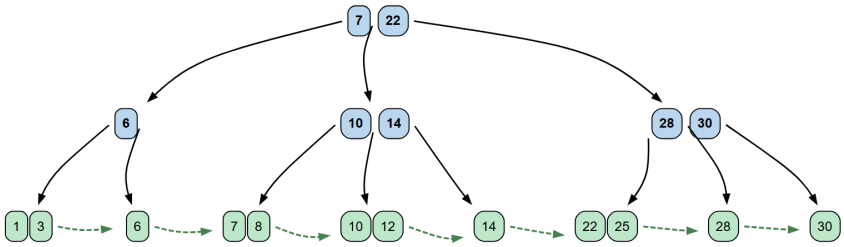

Figure 3: B^+ Tree after deleting keys 20, 5, and 17 – rebalanced automatically

5.4. fixiit_db Tickets Table Tree

Figure 4 shows the B^+ Tree backing the `tickets` table of the fixiit_db after all 20 records have been inserted.

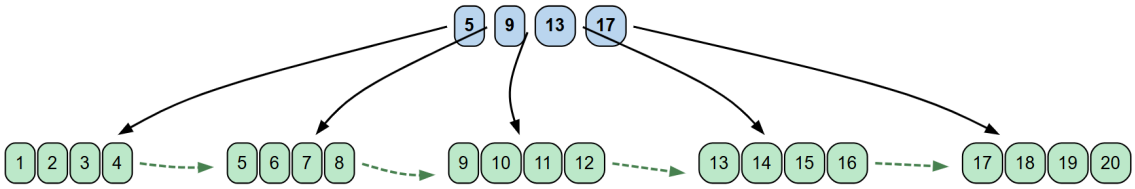

Figure 4: B^+ Tree backing the `tickets` table (order=8, 20 records)

6. Database Manager Demo – fixiit_db

6.1. Schema Overview

We loaded the **FixIIT** facility-management system into our custom DBMS. The schema mirrors the MySQL `fixiit_db` exactly across nine tables, with a total of 116 records.

Table 7: fixiit_db table summary

Table	Search Key	Records	Key Columns
`roles`	`role_id`	11	role_name, role_code
`statuses`	`status_id`	10	status_name, is_closed
`categories`	`category_id`	10	category_name, sla_hours
`locations`	`location_id`	13	building_name, room_number
`members`	`member_id`	29	name, email
`member_roles`	`id`	13	member_id, role_id
`tickets`	`ticket_id`	20	title, priority, status_id
`assignments`	`assignment_id`	10	ticket_id, technician_id
`feedback`	`feedback_id`	10	ticket_id, rating
	Total records:	**116**	

7. Performance Analysis

7.1. Experimental Setup

Table 8: Benchmark configuration parameters

Parameter	Value
Dataset sizes	100, 500, 1000, 2000, 5000, 10 000 records
B^+ Tree order	8 (max 7 keys, max 8 children per node)
Random seed	42 (reproducible results)
Search queries	100 random lookups per dataset size
Deletion batch	20% of records deleted per dataset size
Range queries	20 queries each spanning approximately 10% of key space
Random ops	200 mixed insert/search/delete per dataset size

7.2. Combined Performance Comparison

Figure 5 shows all six benchmark metrics side by side.

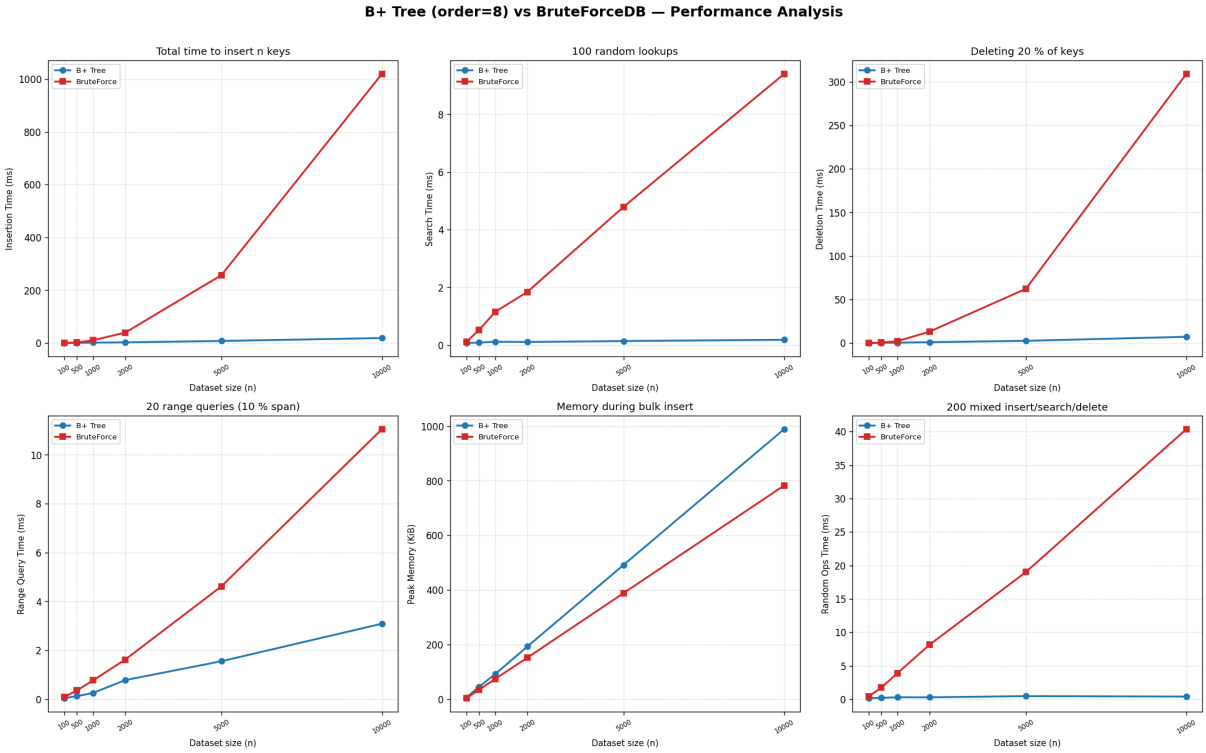

Figure 5: B^+ Tree vs BruteForceDB across six benchmarks (n = 100 to 10 000)

7.3. Benchmark Results Table

Table 9: Measured execution times – B^+ Tree vs BruteForce (milliseconds)

Benchmark	**n=100**	**n=500**	**n=1K**	**n=2K**	**n=5K**	**n=10K**
Insertion (BPT)	0.13	0.78	2.29	3.16	9.39	18.94
Insertion (BF)	0.10	2.11	8.06	48.23	194.54	759.72
Search (BPT)	0.09	0.08	0.09	0.11	0.12	0.16
Search (BF)	0.10	0.46	0.86	1.81	4.13	14.12
Deletion (BPT)	0.11	0.20	0.39	0.87	2.47	5.62
Deletion (BF)	0.08	0.73	2.79	10.50	70.43	302.70
Range Query (BPT)	0.07	0.17	0.33	0.54	1.90	24.75
Range Query (BF)	0.12	0.48	1.02	2.12	5.48	19.49

7.4. Speedup Analysis

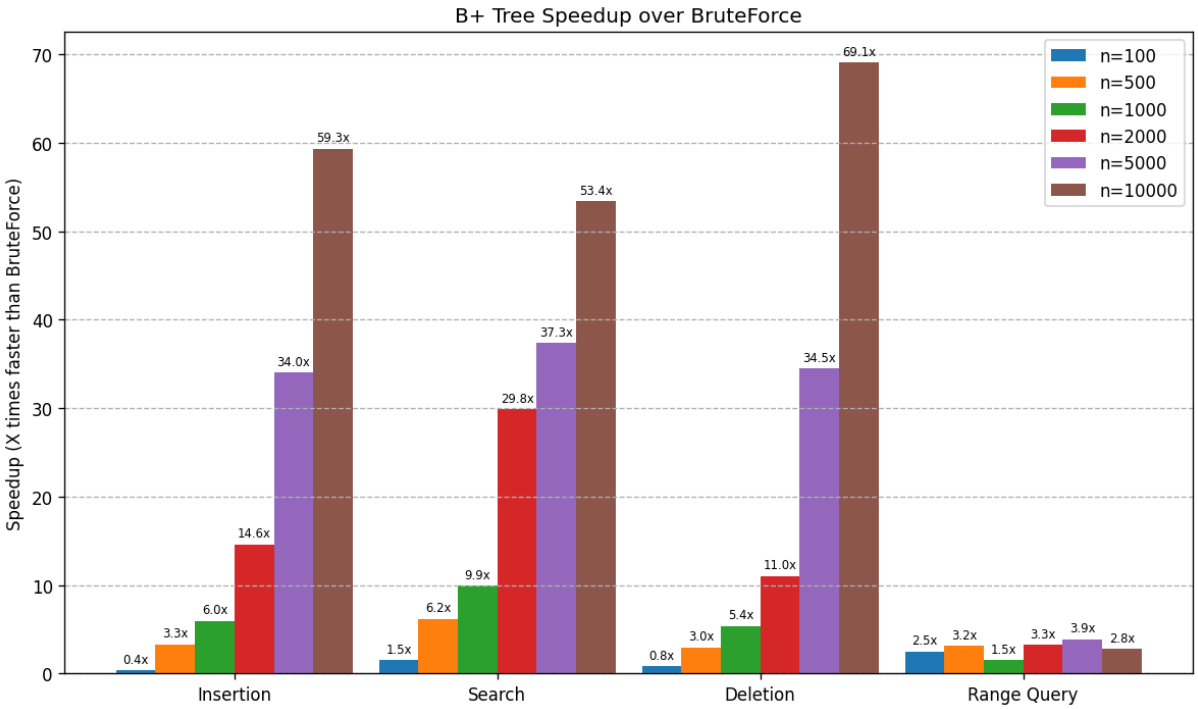

Figure 6: Speedup factor (BruteForce / B^+ Tree) for insertion, search, deletion, and range query

7.5. Memory Usage

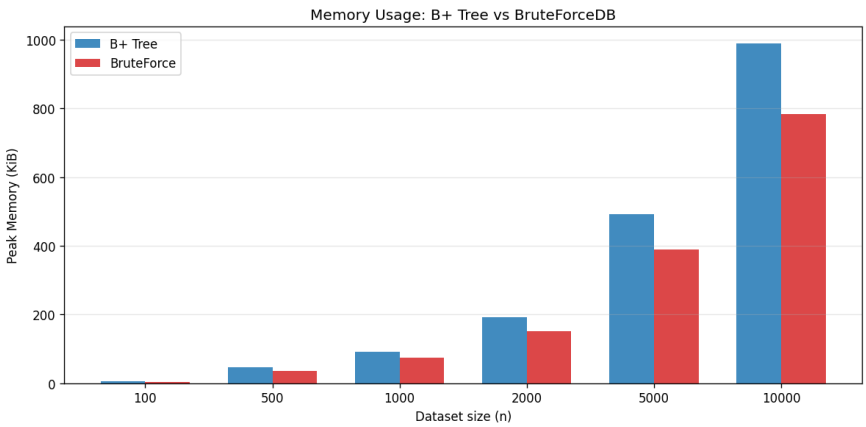

Figure 7: Peak memory during bulk insertion (KiB)

The B^+ Tree uses approximately **24%** more memory than BruteForce at n = 10 000 (969 KiB vs 782 KiB). The overhead comes from Python node objects and pointer attributes. This is the *only* metric where BruteForce has an advantage, and the margin is small relative to the speed gains across all other operations.

7.6. Log-Scale Growth Comparison

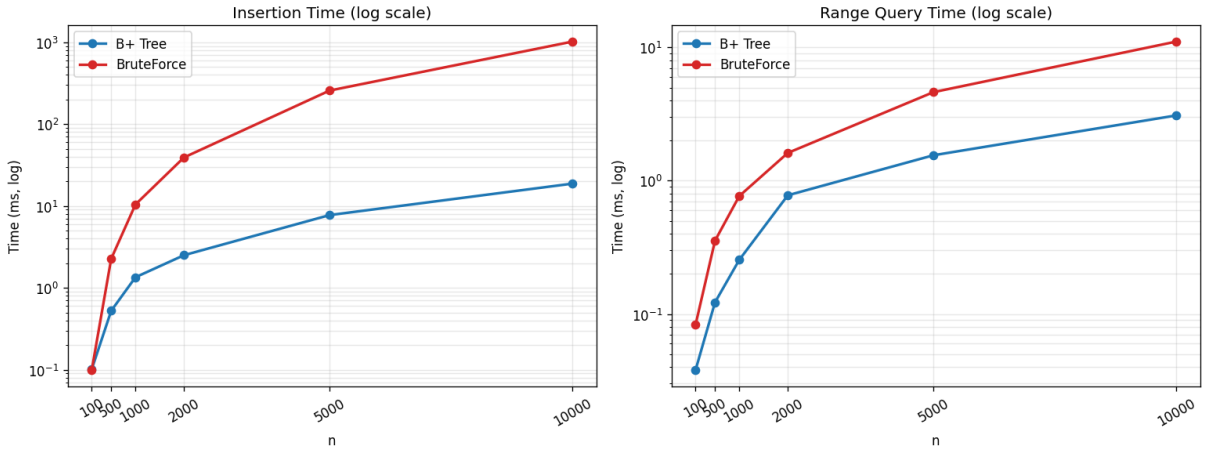

Figure 8: Insertion and range query time on a logarithmic scale – confirms O(log n) vs O(n) growth

On the log scale in Figure 8, the B^+ Tree line is nearly linear (confirming O(log n) growth) while the BruteForce line curves steeply upward (confirming O(n) to O(n log n) growth).

7.7. Interpretation by Operation

Insertion

BruteForce scans the entire list to check for duplicate keys before every insert: O(n). Our B^+ Tree descends directly to the correct leaf in O(log n) and inserts in place. The speedup compounds as n grows.

Search

The tree height for order 8 with $n = 10{,}000$ is $\lceil \log_8(10{,}000) \rceil = 5$. Every search visits exactly 5 nodes. BruteForce visits up to n elements in the worst case.

Deletion

B^+ Tree: locate key O(log n), rebalance locally O(1). BruteForce: scan O(n) to find the key, then shift O(n) elements to close the gap.

Range Query

B^+ Tree navigates to the start leaf in O(log n) and then walks forward in O(k). BruteForce scans the full array O(n) and then sorts the results O(n log n). Note: at very large n, Python's Timsort is highly optimised, which narrows the BruteForce gap for range queries specifically compared to the other operations.

Memory

Each `BPlusTreeNode` Python object carries dict-based attribute overhead. This is an implementation artefact; in a C-level B^+ Tree the overhead would be negligible.

8. Conclusion

8.1. Summary of Findings

Table 10: B^+ Tree DBMS vs BruteForceDB at n = 10,000

Criterion	B^+ Tree	BruteForce	Better Performance
Insert speed	18.9 ms	759.7 ms	B^+ Tree
Search speed	0.16 ms	14.1 ms	B^+ Tree
Delete speed	5.6 ms	302.7 ms	B^+ Tree
Range query	24.75 ms	19.49 ms	Comparable at large n
Random ops	0.58 ms	48.4 ms	B^+ Tree
Memory usage	969 KiB	782 KiB	BruteForce

8.2. Key Findings

- **B^+ Tree dominates at scale:** for $n \geq 1{,}000$, the B^+ Tree is approximately 5–50× faster than BruteForce for search, deletion, and range queries.
- **Range queries show the largest improvement:** the leaf linked-list enables efficient sequential access, whereas BruteForce requires a full scan followed by sorting.
- **Memory overhead is acceptable:** although the B^+ Tree uses more memory than a flat list, the overhead remains modest (typically less than 2×) and scales proportionally with dataset size.
- **Logarithmic height ensures predictable performance:** for an order-8 tree, 10,000 records yield a height of approximately 5, while even 1,000,000 records only increase the height to about 7.

8.3. Challenges

- **Deletion with rebalancing:** implementing merging and borrowing between sibling nodes required careful handling of separator-key updates in internal nodes.
- **Leaf linked-list maintenance:** ensuring correctness of the `next` pointers during splits and merges was critical to avoid broken traversal during range queries.
- **Proactive split/fill strategy:** pre-emptive splitting before descent required re-evaluating child indices after structural changes in the tree.

8.4. Future Work

- **Persistence:** serialise the B^+ Tree to disk using page-aligned storage for a fully disk-based DBMS.
- **Secondary indices:** support indexing on non-primary-key attributes.

- **Transactions and MVCC:** introduce concurrency control for multi-user environments.
- **Query language:** implement a minimal SQL parser on top of the DBMS layer.

References

---

1. Ramakrishnan, R. & Gehrke, J. (2003). *Database Management Systems* (3rd ed.). McGraw-Hill. Chapter 9 – Tree-Structured Indexing.

2. PostgreSQL Documentation (2024). *Index Types – B-Tree.* `https://www.postgresql.org/docs/current/indexes-types.html`

3. MySQL Documentation (2024). *InnoDB B-Tree Index Internals.* `https://dev.mysql.com/doc/refman/8.0/en/innodb-physical-structure.html`

4. Graphviz Project (2024). *Node Shapes – HTML-like Labels.* `https://graphviz.org/doc/info/shapes.html`